

Atividade – Máquinas de Turing e a linguagem 0^n1^n

Etapa 1 – Conceitos

O que é uma Máquina de Turing?

A Máquina de Turing é um modelo matemático de computação proposto por Alan Turing em 1936. Ela não é uma máquina física, e sim uma forma abstrata de representar o que significa "calcular" alguma coisa. A ideia é simples: uma fita infinita dividida em células, uma cabeça que lê e escreve um símbolo por vez e se move para a esquerda ou para a direita, e um conjunto de regras que diz o que fazer em cada situação. Mesmo com essa estrutura tão básica, ela consegue simular qualquer algoritmo que um computador real execute.

Quais são seus principais componentes?

- **Fita:** infinita e dividida em células, cada uma com um símbolo. A entrada começa escrita nela e o resto é preenchido com o símbolo branco (normalmente $_$).
- **Cabeça de leitura/escrita:** fica sobre uma célula por vez, lê o símbolo, pode substituí-lo e depois se move uma posição para a esquerda (L) ou para a direita (R).
- **Conjunto de estados:** funcionam como a "memória" da máquina sobre o ponto em que ela está no processamento. Existe um estado inicial e estados finais (de aceitação e de rejeição).
- **Alfabeto de entrada e alfabeto da fita:** o de entrada contém os símbolos que podem aparecer na palavra analisada. O da fita inclui esses símbolos e mais os auxiliares, como o branco e os marcadores.
- **Função de transição:** as regras no formato (estado atual, símbolo lido) $\rightarrow$ (novo estado, símbolo escrito, direção). É o "programa" da máquina.

Qual é a importância das Máquinas de Turing para a computação?

Ela deu uma definição formal para a ideia de "procedimento computável". Antes disso, algoritmo era um conceito intuitivo. Com a Máquina de Turing, passou a ser possível provar o que pode ou não ser computado, o que fez dela a base da teoria da computabilidade e da complexidade. Além disso, a Máquina de Turing Universal, que simula qualquer outra a partir da descrição dela, inspirou o conceito de computador de programa armazenado que usamos hoje.

Qual é a relação entre Máquina de Turing e algoritmo?

Pela Tese de Church-Turing, tudo o que pode ser resolvido por um algoritmo (um procedimento finito, bem definido e passo a passo) pode ser resolvido por uma Máquina de Turing, e o contrário também vale. Na prática, a Máquina de Turing é a formalização do conceito de algoritmo. Dizer que "existe um algoritmo" para um problema é o mesmo que dizer que existe uma Máquina de Turing que resolve esse problema e sempre para.

Etapa 2 – Configuração da máquina no simulador

Simulador escolhido: [Online Turing Machine Simulator](https://turingmachinesimulator.com) (turingmachinesimulator.com, de Martin Ugarte). Ele roda no navegador, não exige cadastro para executar e cria a máquina a partir de código colado, então não é preciso desenhar estado por estado.

Nesse simulador, a máquina é definida por texto. `init`: indica o estado inicial, `accept`: o estado de aceitação, e cada transição ocupa duas linhas: a primeira com estado, símbolo lido e a segunda com próximo estado, símbolo escrito, movimento. O movimento é `>` (direita), `<` (esquerda) ou `-` (parada). O branco é `_`. A cabeça começa automaticamente no primeiro símbolo da entrada, e o alfabeto é deduzido das transições.

Código da máquina

```
name: Linguagem 0n1n
init: q0
accept: qA
```

```
q0,0
q1,X,>
```

```
q0,Y
q3,Y,>
```

```
q0,1
qR,1,-
```

```
q0,_
qR,-,-
```

```
q1,0
q1,0,>
```

```
q1,Y
q1,Y,>
```

```
q1,1
q2,Y,<
```

```
q1,_
qR,-,-
```

```
q2,0
q2,0,<
```

```
q2,Y
q2,Y,<
```

```
q2,X
q0,X,>
```

```
q3,Y
q3,Y,>
```

q3, _
qA, -, -

q3, 0
qR, 0, -

q3, 1
qR, 1, -

O simulador não tem um campo de "estado de rejeição". Qualquer parada fora de qA é rejeição. Mesmo assim, as transições para qR foram mantidas para que o estado qR apareça na tela no momento da rejeição, o que deixa a captura mais clara.

Tabela de transições

São 15 transições: 10 fazem o trabalho principal e 5 levam explicitamente ao estado de rejeição. Na coluna "Move", R = direita, L = esquerda e S = fica parada.

Nº	Estado atual	Lê	Escreve	Move	Próximo estado	Explicação
1	q0	0	X	R	q1	Marca um 0 e vai procurar o 1 correspondente
2	q0	Y	Y	R	q3	Não há mais 0s; vai verificar o restante
3	q0	1	1	S	qR	1 sem 0 correspondente
4	q0	–	–	S	qR	Palavra vazia
5	q1	0	0	R	q1	Pula os 0s
6	q1	Y	Y	R	q1	Pula os 1s já usados
7	q1	1	Y	L	q2	Marca o 1 e volta
8	q1	–	–	S	qR	Faltou 1 para o 0 marcado

Nº	Estado atual	Lê	Escreve	Move	Próximo estado	Explicação
9	q2	0	0	L	q2	Volta pulando 0s
10	q2	Y	Y	L	q2	Volta pulando Ys
11	q2	X	X	R	q0	Achou o último X; recomeça o ciclo
12	q3	Y	Y	R	q3	Confere que só há Ys
13	q3	–	–	S	qA	Chegou ao fim: aceita
14	q3	0	0	S	qR	0 fora de ordem
15	q3	1	1	S	qR	Sobrou 1 sem par

Combinações que não aparecem na tabela, como q1 lendo X, nunca acontecem durante a execução. Por isso não foram definidas.

Etapas 3 – Registro da simulação

Os três testes foram executados no turingmachinesimulator.com, e o resultado obtido foi igual ao esperado em todos eles. Os estados percorridos foram anotados durante a execução, avançando um passo por vez e registrando o estado exibido a cada passo, incluindo o q0 inicial.

Teste	Entrada	Resultado esperado	Resultado obtido	Estados percorridos
1	0011	ACEITA	ACEITA	q0 → q1 → q1 → q2 → q2 → q0 → q1 → q1 → q2 → q2 → q0 → q3 → q3 → qA
2	000111	ACEITA	ACEITA	q0 → q1 → q1 → q1 → q2 → q2 → q2 → q0 → q1 → q1 → q1 → q2 → q2 → q2 → q0 →

Teste	Entrada	Resultado esperado	Resultado obtido	Estados percorridos
				$q1 \rightarrow q1 \rightarrow q1 \rightarrow$ $q2 \rightarrow q2 \rightarrow q2 \rightarrow$ $q0 \rightarrow q3 \rightarrow q3 \rightarrow$ $q3 \rightarrow qA$
3	00111	REJEITA	REJEITA	$q0 \rightarrow q1 \rightarrow q1 \rightarrow$ $q2 \rightarrow q2 \rightarrow q0 \rightarrow$ $q1 \rightarrow q1 \rightarrow q2 \rightarrow$ $q2 \rightarrow q0 \rightarrow q3 \rightarrow$ $q3 \rightarrow qR$

Capturas de tela da simulação

Teste 1 – entrada 0011: 13 passos, estado final qA, resultado "Accepted".

Linguagem 0n1n

Steps: 13

State: qA

Accepted (show output)

X

X

Y

Y

0011

Load

▶

⏸

■

⏭

Speed:

Examples

Tutorials

Info

1 name: Linguagem 0n1n

2 init: q0

3 accept: qA

4

5 q0,0

6 q1,X,>

7

8 q0,Y

9 q3,Y,>

10

11 q0,1

12 qR,1,-

13

14 q0,_

15 qR,_,-

16

17 q1,0

18 q1,0,>

19

Log in to save

Share Link

Compile

Teste 2 – entrada 000111: 25 passos, estado final qA, resultado "Accepted".

Linguagem 0n1n

Steps: 25

State: qA

Accepted (show output)

X

X

X

Y

Y

Y

000111

Load

▶

⏸

■

⏭

Speed:

Examples

Tutorials

Info

1 name: Linguagem 0n1n

2 init: q0

3 accept: qA

4

5 q0,0

6 q1,X,>

7

8 q0,Y

9 q3,Y,>

10

11 q0,1

12 qR,1,-

13

14 q0,_

15 qR,_,-

16

17 q1,0

18 q1,0,>

19

Log in to save

Share Link

Compile

Teste 3 – entrada 00111: 13 passos, estado final qR, resultado "Rejected".

Linguagem 0n1n

Steps: 13
State: qR
Rejected (show output)

Load

▶
⏸
■
⏭

Speed:

Examples ▾
Tutorials ▾
Info ▾

```

1 name: Linguagem 0n1n
2 init: q0
3 accept: qA
4
5 q0,0
6 q1,X,>
7
8 q0,Y
9 q3,Y,>
10
11 q0,1
12 qR,1,-
13
14 q0,_
15 qR,-,-
16
17 q1,0
18 q1,0,>
19

```

Log in to save
Share Link

Compile

Interpretação dos resultados

Nos testes 1 e 2, a máquina terminou com a fita toda marcada (XXYY e XXXYYY). Isso mostra que cada 0 encontrou um 1 correspondente e não sobrou nenhum símbolo sem par, por isso ela chegou em qA.

No teste 3, a máquina formou dois pares, foi para q3 conferir o restante da fita e parou sobre um 1 sem marca (fita final XXYY1). Esse 1 a mais é exatamente o que faz a palavra não pertencer à linguagem, e a máquina foi para qR.

Também dá para perceber o custo da estratégia de ir e voltar na fita. A palavra 01, usada no teste de treino, levou 5 passos, 0011 levou 13 e 000111 levou 25. Cada par novo exige uma viagem mais longa, então o número de passos cresce aproximadamente com o quadrado do tamanho da entrada.

Descrição da Máquina de Turing criada

A máquina reconhece a linguagem $L = \{0^n1^n \mid n \geq 1\}$: palavras com uma sequência de 0s seguida da mesma quantidade de 1s. Ela aceita 01, 0011, 000111 e 00001111, e rejeita 0, 1, 001, 011 e 00111.

O funcionamento é baseado em ciclos. A cada ciclo, a máquina marca o primeiro 0 disponível com **X**, anda para a direita até o primeiro 1 disponível, marca esse 1 com **Y** e volta até o último X para começar de novo. Quando não sobra nenhum 0, ela entra em modo de verificação e confere se o restante da fita tem apenas Ys. A palavra é aceita só se todos os símbolos foram pareados. Ela é rejeitada se sobrar um 0 sem par, um 1 sem par, ou se os símbolos estiverem fora de ordem.

Os símbolos X e Y são a forma de a máquina "lembrar" o que já foi usado, já que ela não tem outra memória além dos estados e da própria fita. Essa linguagem é um exemplo clássico porque não é regular: um autômato finito não consegue contar uma quantidade ilimitada de 0s, mas a Máquina de Turing resolve isso usando a fita como memória.

Item	Definição
Estados	q0, q1, q2, q3, qA, qR
Estado inicial	q0
Estado de aceitação	qA
Estado de rejeição	qR
Alfabeto de entrada	{0, 1}
Alfabeto da fita	{0, 1, X, Y, _}
Símbolo branco	—
Posição inicial da cabeça	Primeiro símbolo da entrada (mais à esquerda)

Significado de cada estado

Estado	Papel
q0	Início de ciclo. Se lê 0, marca com X e vai procurar o 1. Se lê Y, todos os 0s já foram usados e vai para a verificação final.
q1	Procura um 1. Anda para a direita pulando 0s e Ys até achar o primeiro 1 livre, que vira Y.
q2	Volta. Anda para a esquerda pulando 0s e Ys até achar um X, dá um passo para a direita e volta para q0.
q3	Verificação final. Anda para a direita confirmando que só há Ys até o branco.
qA	Aceitação: a palavra pertence à linguagem.
qR	Rejeição: a palavra não pertence à linguagem.

A palavra vazia foi tratada como rejeitada ($n \geq 1$), já que os exemplos do enunciado começam em 01. Se o professor considerar $n \geq 0$, basta trocar a transição ($q_0, _$) para ir para q_A .

Etapa 4 – Reflexão sobre os limites computacionais

Não. Mesmo sendo o modelo mais geral de computação que conhecemos, a Máquina de Turing não resolve todos os problemas. Existem problemas para os quais nenhum algoritmo consegue dar a resposta certa em todos os casos. O exemplo clássico é o Problema da Parada: decidir, para qualquer programa e entrada, se ele termina ou roda para sempre. Turing provou que isso é impossível, porque um programa que consultasse esse "decisor" sobre si mesmo e fizesse o contrário do previsto geraria uma contradição. Além disso, os algoritmos são finitos e podem ser listados, mas os problemas possíveis são muito mais numerosos, então sempre sobram problemas sem solução. Esse limite não é falta de tempo ou de memória, e sim uma limitação lógica da própria ideia de algoritmo.

Questão final: problema difícil ou sem algoritmo?

A diferença está em existir ou não um algoritmo que sempre para com a resposta certa. Num problema difícil ele existe, mas é caro demais. Num problema indecidível ele não existe.

Problema computacionalmente difícil: existe um algoritmo que resolve o problema e sempre para com a resposta certa. A questão é o custo, porque o tempo ou a memória crescem muito rápido conforme a entrada aumenta, às vezes de forma exponencial. Exemplos são o Problema do Caixeiro Viajante e o SAT, que são NP-completos. Uma Máquina de Turing consegue resolvê-los, mas para entradas grandes a execução pode levar mais tempo do que é viável. Isso é estudado pela **teoria da complexidade** (classes P, NP, NP-completo).

Problema indecidível ou não computável: não existe nenhuma Máquina de Turing que resolva o problema corretamente e pare para todas as entradas. Não importa quanto tempo, memória ou poder de processamento se tenha, porque o limite é lógico e não de desempenho. Exemplos são o Problema da Parada e, pelo Teorema de Rice, qualquer pergunta não trivial sobre o comportamento de programas, como "este programa sempre retorna 0?". Isso é estudado pela **teoria da computabilidade**.

Como descobrir em qual caso o problema se encaixa

1. **Tentar construir um algoritmo que sempre para.** Se é possível descrever um procedimento, mesmo lento (como uma força bruta que testa todas as possibilidades), e garantir que ele termina para qualquer entrada, o problema é decidível. A partir daí a questão é só de eficiência.
2. **Usar redução para mostrar que é difícil.** Se um problema NP-completo conhecido pode ser transformado no meu problema em tempo polinomial, o meu problema é pelo menos tão difícil quanto ele. Provavelmente não há solução eficiente, mas ainda há solução.
3. **Usar redução para mostrar que é indecidível.** Se um algoritmo para o meu problema permitiria resolver o Problema da Parada, então o meu problema também é indecidível, já que o Problema da Parada comprovadamente não tem solução.
4. **Verificar o Teorema de Rice.** Se o problema pergunta sobre uma propriedade não trivial do comportamento de um programa, e não sobre o código em si, ele já é indecidível.

Resumindo: num problema difícil a Máquina de Turing chega na resposta, mas pode demorar demais. Num problema indecidível, nenhuma Máquina de Turing tem garantia de chegar na resposta. Na prática, isso muda a estratégia. Para problemas difíceis, faz sentido buscar heurísticas, aproximações ou soluções para casos específicos. Para problemas indecidíveis, o caminho é restringir o problema (por exemplo, analisar só uma classe limitada de programas) ou aceitar respostas do tipo "não sei".